Sunline Filter

Component Design Document

1 Description

Sunline filter. Estimates the sun direction in the body frame, the body rate, and the coarse sun sensor intensity bias from the coarse sun sensor cosines and the body rate with a square root unscented Kalman filter, and publishes the estimate for the guidance algorithms. The filter state, the variance of each state, and the residuals of each measurement are published as data products. The filter synchronizes its time to the first tick after it is built or reset and propagates from there, and takes the time of each measurement from its data product. Wraps the SunlineFilterAlgorithm C++ algorithm via its C shim.

2 Requirements

No requirements have been specified for this component.

3 Design

3.1 At a Glance

Below is a list of useful parameters and statistics that give a quick look into the makeup of the component.

- **Execution** - *passive*
- **Number of Connectors** - 6
- **Number of Invokee Connectors** - 4
- **Number of Invoker Connectors** - 2
- **Number of Generic Connectors** - *None*
- **Number of Generic Types** - *None*
- **Number of Unconstrained Arrayed Connectors** - *None*
- **Number of Commands** - *None*
- **Number of Parameters** - 13
- **Number of Events** - *None*
- **Number of Faults** - *None*
- **Number of Data Products** - 5
- **Number of Data Dependencies** - 2
- **Number of Packets** - *None*

3.2 Diagram

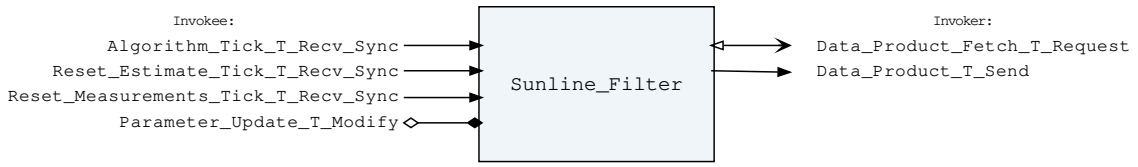


Figure 1: Sunline Filter component diagram.

3.3 Connectors

Below are tables listing the component's connectors.

3.3.1 Invokee Connectors

The following is a list of the component's *invokee* connectors:

Table 1: Sunline Filter Invokee Connectors

Name	Kind	Type	Return_Type	Count
Algorithm_Tick_T_Recv_Sync	recv_sync	Algorithm_Tick.T	-	1
Reset_Estimate_Tick_T_Recv_Sync	recv_sync	Tick.T	-	1
Reset_Measurements_Tick_T_Recv_Sync	recv_sync	Tick.T	-	1
Parameter_Update_T_Modify	modify	Parameter_Update.T	-	1

Connector Descriptions:

- **Algorithm_Tick_T_Recv_Sync** - Run the filter up to the tick's call time, folding in the coarse sun sensor and body rate readings that are new since the last tick.
- **Reset_Estimate_Tick_T_Recv_Sync** - Re-seed the filter state and covariance from the configured initial values and clear the pending measurements and residuals. The filter synchronizes its time again at the next tick. The assembly fires this on every entry to the navigation state, where the filter is first put into use.
- **Reset_Measurements_Tick_T_Recv_Sync** - Clear the pending measurements and residuals, keeping the filter state and covariance. The filter synchronizes its time again at the next tick. The assembly fires this on every transition between pointing states.
- **Parameter_Update_T_Modify** - The parameter update connector.

3.3.2 Invoker Connectors

The following is a list of the component's *invoker* connectors:

Table 2: Sunline Filter Invoker Connectors

Name	Kind	Type	Return_Type	Count
Data_Product_Fetch_T_Request	request	Data_Product_Fetch.T	Data_Product_Return.T	1

Data_Product_T_Send	send	Data_Product.T	-	1

Connector Descriptions:

- **Data_Product_Fetch_T_Request** - Fetch a data product item from the database.
- **Data_Product_T_Send** - The data product invoker connector

3.4 Interrupts

This component contains no interrupts.

3.5 Initialization

Below are details on how the component should be initialized in an assembly.

3.5.1 Component Instantiation

This component contains no instantiation parameters in its discriminant.

3.5.2 Component Base Initialization

This component contains no base class initialization, meaning there is no `init_Base` subprogram for this component.

3.5.3 Component Set ID Bases

This component contains commands, events, packets, faults, or data products that require a base identifier to be set at initialization. The `set_Id_Bases` procedure must be called with the following parameters:

Table 3: Sunline Filter Set Id Bases Parameters

Name	Type
Data_Product_Id_Base	Data_Product_Types.Data_Product_Id_Base
Parameter_Id_Base	Parameter_Types.Parameter_Id_Base

Parameter Descriptions:

- **Data_Product_Id_Base** - The value at which the component's data product identifiers begin.
- **Parameter_Id_Base** - The value at which the component's parameter identifiers begin.

3.5.4 Component Map Data Dependencies

This component contains data dependencies that need their IDs mapped to valid data product IDs at initialization. Each data dependency also needs a stale limit provided in microseconds. If the timestamp of the data dependency is older than the reference time (usually the current time) minus the stale limit then the data dependency is considered too old to safely use. The `map_Data_Dependencies` procedure must be called with the following parameters:

Table 4: Sunline Filter Map Data Dependencies Parameters

Name	Type
Body_Rate_Id	Data_Product_Types.Data_Product_Id

Body_Rate_Stale_Limit	Ada.Real_Time.Time_Span
Css_Cosines_Id	Data_Product_Types.Data_Product_Id
Css_Cosines_Stale_Limit	Ada.Real_Time.Time_Span

Parameter Descriptions:

- **Body_Rate_Id** - The external data product ID used to fetch the value for the 'Body_Rate' data dependency.
- **Body_Rate_Stale_Limit** - The stale limit used for the 'Body_Rate' data dependency. If the fetched data product is older than the given Time_Span then it is deemed stale and unfit for use by the component.
- **Css_Cosines_Id** - The external data product ID used to fetch the value for the 'Css_Cosines' data dependency.
- **Css_Cosines_Stale_Limit** - The stale limit used for the 'Css_Cosines' data dependency. If the fetched data product is older than the given Time_Span then it is deemed stale and unfit for use by the component.

3.5.5 Component Implementation Initialization

The calling of this implementation class initialization procedure is mandatory. Initializes the sunline filter with the default parameter values, which seed the filter state and covariance. The `init` subprogram takes no parameters.

3.6 Commands

The Sunline Filter component has no commands.

3.7 Parameters

Parameters for the Sunline Filter component. The process noise and the initial covariance are given by their diagonals, since the full matrices do not fit in a parameter.

Table 5: Sunline Filter Parameters

Local ID	Parameter Name	Type	Default Value
0x0000 (0)	Alpha	Packed_F64.T	(Value=>0.02)
0x0001 (1)	Beta	Packed_F64.T	(Value=>2.0)
0x0002 (2)	Process_Noise_Diagonal	Packed_F64x7.T	[1.0E-12, 1.0E-12, 1.0E-12, 1.0E-16, 1.0E-16, 1.0E-16,1.0E-10]
0x0003 (3)	Initial_State	Packed_F64x7.T	[0.0,0.0,1.0, 0.0,0.0,0.0, 1.0]
0x0004 (4)	Initial_Covariance_Diagonal	Packed_F64x7.T	[0.1,0.1,0.1, 0.001,0.001, 0.001,0.5]
0x0005 (5)	Bias_Lower_Bound	Packed_F64.T	(Value=>0.5)
0x0006 (6)	Bias_Upper_Bound	Packed_F64.T	(Value=>1.5)

0x0007 (7)	Css_N_Hat_B	Packed_ F32x24.T	[+0.70710678, -0.50000000, +0.50000000, +0.70710678, -0.50000000, -0.50000000, +0.70710678, +0.50000000, -0.50000000, +0.70710678, +0.50000000, +0.50000000, -0.70710678, +0.00000000, +0.70710678, -0.70710678, +0.70710678, +0.00000000, -0.70710678, +0.00000000, -0.70710678, -0.70710678, -0.70710678, +0.00000000]
0x0008 (8)	Css_Scale_Factor	Packed_F64x8. T	[1.0,1.0,1.0, 1.0,1.0,1.0, 1.0,1.0]
0x0009 (9)	Sensor_Threshold	Packed_F64.T	(Value=>0.0)
0x000a (10)	Css_Measurement_Noise_Std	Packed_F64.T	(Value=>0.01)
0x000b (11)	Gyro_Measurement_Noise_Std	Packed_F64.T	(Value=>0. 001)
0x000c (12)	Measurement_Delay	Packed_F64.T	(Value=>0.0)

Parameter Descriptions:

- **Alpha** - [-] Sigma point spread. Must be in (0, 1].
- **Beta** - [-] Prior knowledge tunable. Must be in [0, 2].
- **Process_Noise_Diagonal** - [-] Diagonal of the N x N process noise covariance, one entry per state: the sun direction, the body rate, and the intensity bias. Every entry must be finite and not negative.
- **Initial_State** - [-] State the filter is seeded with: the sun direction in the body frame, the body rate in rad/s, and the intensity bias. Must be finite. The seed is taken when the filter is built and at the estimate reset; a parameter update alone keeps the running estimate.
- **Initial_Covariance_Diagonal** - [-] Diagonal of the N x N covariance the filter is seeded with, one entry per state. Every entry must be finite and not negative. The seed is taken when the filter is built and at the estimate reset; a parameter update alone keeps the running estimate.
- **Bias_Lower_Bound** - [-] Lower clamp on the intensity bias state. Must be greater than zero and less than the upper bound.
- **Bias_Upper_Bound** - [-] Upper clamp on the intensity bias state. Must be greater than the lower bound.
- **Css_N_Hat_B** - [-] Boresight unit vector of each coarse sun sensor slot in body frame components, three per sensor in row major order. Each must be a unit vector to within 1e-3.

- **Css_Scale_Factor** - [-] Calibration scale factor of each coarse sun sensor slot. One means the nominal response. Every entry must not be negative.
- **Sensor_Threshold** - [-] Cosine value above which a sensor counts as active in a measurement. Must not be negative.
- **Css_Measurement_Noise_Std** - [-] Standard deviation of the coarse sun sensor measurement noise. Must not be negative.
- **Gyro_Measurement_Noise_Std** - [rad/s] Standard deviation of the gyro measurement noise. Must not be negative.
- **Measurement_Delay** - [s] Processing delay for late readings. A reading is applied only once the filter time has reached its time tag plus the delay, so a reading of one kind that arrives later than the other can still be applied in time order. Zero applies every reading at once. Must not be negative.

3.8 Events

The Sunline Filter component has no events.

3.9 Data Products

Data products for the Sunline Filter component.

Table 6: Sunline Filter Data Products

Local ID	Data Product Name	Type
0x0000 (0)	Sun_Direction_Estimate	Nav_Att_Output.T
0x0001 (1)	Filter_State	Sunline_Filter_State.T
0x0002 (2)	Css_Pre_Fit_Residuals	Sunline_Filter_Css_Fit_Residuals.T
0x0003 (3)	Css_Post_Fit_Residuals	Sunline_Filter_Css_Fit_Residuals.T
0x0004 (4)	Rate_Residuals	Sunline_Filter_Rate_Fit_Residuals.T

Data Product Descriptions:

- **Sun_Direction_Estimate** - Estimated sun direction in the body frame and body rate, stamped with the time the filter advanced to. The attitude is not estimated by this filter and is zero.
- **Filter_State** - The filter state after the update, the variance of each state, and the system time of the last coarse sun sensor and body rate readings applied.
- **Css_Pre_Fit_Residuals** - Coarse sun sensor residuals before this update, with whether that measurement fired and how many sensors were active.
- **Css_Post_Fit_Residuals** - Coarse sun sensor residuals after this update, with whether that measurement fired and how many sensors were active.
- **Rate_Residuals** - Gyro rate residuals of this update, with whether that measurement fired.

3.10 Data Dependencies

Data dependencies for the Sunline Filter component. Each product is fed to the filter as a measurement when its timestamp has changed since the last one consumed, so the timestamp is the measurement time.

Table 7: Sunline Filter Data Dependencies

Data Dependency Name	Type
Body_Rate	Packed_F32x3.T
Css_Cosines	Css_Sensor_Values.T

Data Dependency Descriptions:

- **Body_Rate** - [rad/s] Selected body rate in body frame components, the gyro measurement of the filter.
- **Css_Cosines** - [-] Corrected cosine value of each coarse sun sensor slot, the sun sensor measurement of the filter.

3.11 Packets

The Sunline Filter component has no packets.

3.12 Faults

The Sunline Filter component has no faults.

4 Unit Tests

The following section describes the unit test suites written to test the component.

4.1 *Sunline_Filter_Tests* Test Suite

This is a unit test suite for the Sunline Filter component

Test Descriptions:

- **Test** - Feed the sun sensor cosines and body rate of a slow rotation and check the sun direction, rate, and bias estimates converge to the truth while the covariance shrinks, as in the algorithm's Python reference test, to ensure the Ada to C to C++ integration is sound. The filter state and residual products report the updates and their times.
- **Test_Propagation** - Ensure products whose timestamps have not advanced are not fed to the filter, so the sun direction is propagated with the seeded rate and the covariance grows under the process noise.
- **Test_Reading_Before_Sync** - Ensure a product not refreshed since before a reset is not consumed again, products stamped before the tick the filter synchronized its time to are consumed and dropped, and the ones after it are applied.
- **Test_Time_Anomalies** - Ensure products stamped after the tick are applied at once and not again when the tick reaches them, without shutting out the readings that follow, and products stale by their stamp are still read.
- **Test_Sensor_Threshold** - Ensure only the sensors reading above the threshold count as active.
- **Test_Reset** - Ensure the estimate reset re-seeds the state and covariance while the measurements reset keeps them.
- **Test_Parameter_Update** - Ensure a parameter update reaches the filter while keeping the current estimate.
- **Test_Measurement_Delay** - Ensure a configured measurement delay holds a reading until the filter time has reached its time tag plus the delay, then applies it.

- **Test_Invalid_Parameter** - Ensure a staged configuration the algorithm would reject is refused at validation.
- **Test_Invalid_Data_Dependency** - Ensure a data dependency with the wrong identifier is treated as a wiring defect and fails the tick's assertion.

5 Appendix

5.1 Preamble

This component contains no preamble code.

5.2 Packed Types

The following section outlines any complex data types used in the component in alphabetical order. This includes packed records and packed arrays that might be used as connector types, command arguments, event parameters, etc..

Algorithm_Tick.T:

The tick an algorithm component runs on, with the time the algorithm propagates to already converted to nanoseconds. The component that dispatches the algorithm ticks converts the tick time once and hands the same value to every algorithm it runs in that tick, so the clock the algorithms propagate on is chosen in exactly one place.

Table 8: Algorithm_Tick Packed Record : 144 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Current_Tick	Tick.T	-	80	0	79
Call_Time	Interfaces. Unsigned_64	0 to 18446744073709551615	64	80	143

Field Descriptions:
- **Current_Tick** - The tick as scheduled. Algorithm components stamp their data products with its time and use it as the stale reference for their data dependencies.
- **Call_Time** - [ns] The time the algorithm propagates to, in nanoseconds of the tick time.

Css_Sensor_Values.T:

Corrected CSS sensor cosine values for the 8 physical CSS ADC channels.

Table 9: Css_Sensor_Values Packed Record : 512 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Data	Packed_F64x8.T	-	512	0	511

Field Descriptions:
- **Data** - *No description provided.*

Data_Product.T:

Generic data product packet for holding arbitrary data types

Table 10: Data_Product Packed Record : 1096 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Data_Product_ Header.T	-	72	0	71	-
Buffer	Data_Product_ Types.Data_ Product_ Buffer_Type	-	1024	72	1095	Header. Buffer_Length

Field Descriptions:

- **Header** - The data product header
- **Buffer** - A buffer that contains the data product type

Data_Product_Fetch.T:

A packed record which holds information for a data product request.

Table 11: Data_Product_Fetch Packed Record : 16 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Id	Data_Product_Types. Data_Product_Id	0 to 65535	16	0	15

Field Descriptions:

- **Id** - The data product identifier

Data_Product_Header.T:

Generic data_product packet for holding arbitrary data_product types

Table 12: Data_Product_Header Packed Record : 72 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	48	0	47
Id	Data_Product_Types. Data_Product_Id	0 to 65535	16	48	63
Buffer_Length	Data_Product_ Types.Data_Product_ Buffer_Length_Type	0 to 128	8	64	71

Field Descriptions:

- **Time** - The timestamp for the data product item.
- **Id** - The data product identifier
- **Buffer_Length** - The number of bytes used in the data product buffer

Data_Product_Return.T:

This record holds data returned from a data product fetch request.

Table 13: Data_Product_Return Packed Record : 1104 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
The_ Status	Data_ Product_ Enums. Fetch_ Status.E	0 => Success 1 => Not_Available 2 => Id_Out_Of_Range	8	0	7	-
The_ Data_ Product	Data_ Product. T	-	1096	8	1103	-

Field Descriptions:

- **The_Status** - A status relating whether or not the data product fetch was successful or not.
- **The_Data_Product** - The data product item returned.

Nav_Att_Output.T:

Table 14: Nav_Att_Output Packed Record : 352 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time_Tag	Long_Float	-inf to inf	64	0	63
Sigma_Bn	Packed_F32x3.T	-	96	64	159
Omega_Bn_B	Packed_F32x3.T	-	96	160	255
Veh_Sun_Pnt_Bdy	Packed_F32x3.T	-	96	256	351

Field Descriptions:

- **Time_Tag** - [s] Time tag of navigation solution
- **Sigma_Bn** - MRP attitude of body w.r.t. inertial frame
- **Omega_Bn_B** - [rad/s] Angular velocity of body w.r.t. inertial, body frame components
- **Veh_Sun_Pnt_Bdy** - Sun direction unit vector in body frame

Packed_F32x24.T:

Packed 24 element vector of 32-bit floats. Holds a table of 8 three-component vectors in row major order.

Table 15: Packed_F32x24 Packed Array : 768 bits

Type	Range	Element Size (Bits)	Length	Total Size (Bits)
Short_Float	-3.40282e+38 to 3.40282e+38	32	24	768

Packed_F32x3.T:

Packed 3 element vector of 32-bit floats.

Table 16: Packed_F32x3 Packed Array : 96 bits

Type	Range	Element Size (Bits)	Length	Total Size (Bits)
Short_Float	-3.40282e+38 to 3.40282e+38	32	3	96

Packed_F64.T:

Single component record for holding packed 64-bit floating point number.

Table 17: Packed_F64 Packed Record : 64 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Value	Long_Float	-inf to inf	64	0	63

Field Descriptions:

- **Value** - The 64-bit floating point number.

Packed_F64x3.T:

Packed 3 element vector of 64-bit floats.

Table 18: Packed_F64x3 Packed Array : 192 bits

Type	Range	Element Size (Bits)	Length	Total Size (Bits)
Long_Float	-inf to inf	64	3	192

Packed_F64x7.T:

Packed 7 element vector of 64-bit floats.

Table 19: Packed_F64x7 Packed Array : 448 bits

Type	Range	Element Size (Bits)	Length	Total Size (Bits)
Long_Float	-inf to inf	64	7	448

Packed_F64x8.T:

Packed 8 element vector of 64-bit floats.

Table 20: Packed_F64x8 Packed Array : 512 bits

Type	Range	Element Size (Bits)	Length	Total Size (Bits)
Long_Float	-inf to inf	64	8	512

Parameter.T:

Generic parameter packet for holding a generic parameter

Table 21: Parameter Packed Record : 792 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Parameter_ Header.T	-	24	0	23	-
Buffer	Parameter_ Types. Parameter_ Buffer_Type	-	768	24	791	Header.Buffer_ Length

Field Descriptions:

- **Header** - The parameter header
- **Buffer** - A buffer that contains the parameter type

Parameter_Header.T:

Generic parameter header for holding arbitrary parameters

Table 22: Parameter_Header Packed Record : 24 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Id	Parameter_Types. Parameter_Id	0 to 65535	16	0	15
Buffer_Length	Parameter_Types. Parameter_Buffer_ Length_Type	0 to 96	8	16	23

Field Descriptions:

- **Id** - The parameter identifier
- **Buffer_Length** - The number of bytes used in the parameter type buffer

Parameter_Update.T:

A record intended to be used as a provide/modify connector type for updating/fetching parameters.

Table 23: Parameter_Update Packed Record : 824 bits *(maximum)*

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Table_Id	Parameter_ Types. Parameter_ Table_Id	0 to 65535	16	0	15	-
Operation	Parameter_ Enums. Parameter_ Operation_ Type.E	0 => Stage 1 => Update 2 => Fetch 3 => Validate	8	16	23	-

Status	Parameter_Enums.Parameter_Update_Status.E	0 => Success 1 => Id_Error 2 => Validation_Error 3 => Length_Error	8	24	31	-
Param	Parameter.T	-	792	32	823	-

Field Descriptions:

- **Table_Id** - The ID for the table that contains this parameter
- **Operation** - The parameter operation to perform.
- **Status** - The parameter return status.
- **Param** - The parameter that has been updated or fetched.

Sunline_Filter_Css_Fit_Residuals.T:

Coarse sun sensor residuals of one sunline filter update, either before or after the update. Built from the residuals the sunlineFilter C shim returns, without the measured cosines, which are already published by the sensor product.

Table 24: Sunline_Filter_Css_Fit_Residuals Packed Record : 552 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Valid	Boolean	0 => False 1 => True	8	0	7
Number_Of_Active_Css	Interfaces.Integer_32	-2147483648 to 2147483647	32	8	39
Residuals	Packed_F64x8.T	-	512	40	551

Field Descriptions:

- **Valid** - Whether the coarse sun sensor measurement fired and was applied this update. The other fields are meaningful only when it did.
- **Number_Of_Active_Css** - [-] Number of sensors above the sensor threshold. Only that many entries of the residuals are meaningful.
- **Residuals** - [-] Residual of each sensor slot.

Sunline_Filter_Rate_Fit_Residuals.T:

Gyro rate residuals of one sunline filter update, before and after the update. Built from the residuals the sunlineFilter C shim returns, without the measured rate, which is already published by the body rate product.

Table 25: Sunline_Filter_Rate_Fit_Residuals Packed Record : 392 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Valid	Boolean	0 => False 1 => True	8	0	7
Pre_Fit	Packed_F64x3.T	-	192	8	199

Post_Fit	Packed_F64x3.T	-	192	200	391

Field Descriptions:

- **Valid** - Whether the rate measurement fired and was applied this update. The residuals are meaningful only when it did.
- **Pre_Fit** - [-] Residual before the update.
- **Post_Fit** - [-] Residual after the update.

Sunline_Filter_State.T:

State of the sunline filter after an update, the estimated state, the variance of each state, and when each kind of reading was last consumed. Built from the snapshot the sunlineFilter C shim returns, with the N x N covariance reduced to its diagonal.

Table 26: Sunline_Filter_State Packed Record : 1024 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
State	Packed_F64x7.T	-	448	0	447
Covariance_Diagonal	Packed_F64x7.T	-	448	448	895
Last_Css_Measurement_Time	Interfaces.Unsigned_64	0 to 18446744073709551615	64	896	959
Last_Rate_Measurement_Time	Interfaces.Unsigned_64	0 to 18446744073709551615	64	960	1023

Field Descriptions:

- **State** - [-] Estimated state, the sun direction in the body frame, the body rate in rad/s, and the intensity bias.
- **Covariance_Diagonal** - [-] Diagonal of the state covariance, the variance of each state.
- **Last_Css_Measurement_Time** - [ns] System time of the last coarse sun sensor reading consumed by the filter, zero until one arrives after the filter is built or reset. Tells when the sun sensors last fed the filter once this product is sampled into a packet.
- **Last_Rate_Measurement_Time** - [ns] System time of the last body rate reading consumed by the filter, zero until one arrives after the filter is built or reset. Tells when the body rate last fed the filter once this product is sampled into a packet.

Sys_Time.T:

A record which holds a time stamp using EMA VTC format including seconds and subseconds since epoch (1-5-1980 to 1-6-1980 midnight).

Table 27: Sys_Time Packed Record : 48 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Seconds	Interfaces.Unsigned_32	0 to 4294967295	32	0	31

Subseconds	Interfaces. Unsigned_16	0 to 65535	16	32	47

Field Descriptions:

- **Seconds** - The number of seconds elapsed since epoch.
- **Subseconds** - The number of 1/(2^16) sub-seconds.

Tick.T:

The tick datatype used for periodic scheduling. Included in this type is the Time associated with a tick and a count.

Table 28: Tick Packed Record : 80 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	48	0	47
Count	Interfaces. Unsigned_32	0 to 4294967295	32	48	79

Field Descriptions:

- **Time** - The timestamp associated with the tick.
- **Count** - The cycle number of the tick.

5.3 Enumerations

The following section outlines any enumerations used in the component.

Data_Product_Enums.Fetch_Status.E:

This status denotes whether a data product fetch was successful.

Table 29: Fetch_Status Literals:

Name	Value	Description
Success	0	The data product was returned successfully.
Not_Available	1	No data product is yet available for the provided id.
Id_Out_Of_Range	2	The data product id was out of range.

Parameter_Enums.Parameter_Operation_Type.E:

This enumeration lists the different parameter operations that can be performed.

Table 30: Parameter_Operation_Type Literals:

Name	Value	Description
Stage	0	Stage the parameter.
Update	1	All parameters are staged, it is ok to update all parameters now.
Fetch	2	Fetch the parameter.
Validate	3	Validate all the parameters.

Parameter_Enums.Parameter_Update_Status.E:

This status enumeration provides information on the success/failure of a parameter operation.

Table 31: Parameter_Update_Status Literals:

Name	Value	Description
Success	0	Parameter was successfully staged.
Id_Error	1	Parameter id was not valid.
Validation_Error	2	Parameter values were not successfully validated.
Length_Error	3	Parameter length was not correct.